NOI.PH Training: Bootcamp

Disjoint Sets, and Union-Find

Kevin Charles Atienza

Contents

1 Introduction

We discuss a new data structure which we'll call **Disjoint Sets**.

As discussed in a previous lesson, we would like to distinguish between the *data structure* and the abstract *data type*. (You may want to review that if needed.) The abstract data type is a description of the new data type—what it is, and what the operations are—while the data structure is how the abstract data type is implemented in your program behind the scenes.

1.1 The Disjoint Sets data type

The Disjoint Sets *data type* consists of n objects, usually just the integers 0 to $n-1$, partitioned into several *disjoint* sets. "Disjoint" simply means no two sets intersect—in other words, every object belongs to only one set. For example, at one point in time, the state of the data type with $n = 7$ could be any of these:

- $[\{0\}, \{1\}, \{2\}, \{3\}, \{4\}, \{5\}, \{6\}]$,
- $[\{0, 1, 2, 3, 4, 5, 6\}]$,
- $[\{0, 1, 5\}, \{2, 4\}, \{3\}, \{6\}]$,
- $[\{0, 3, 4, 5, 6\}, \{1, 2\}]$,
- etc.

At the beginning, the state of the data type consists of n sets, each containing a single element, just like the first example above. The data type supports these two basic operations:

- $union(i, j)$. Given two objects i and j, merge the sets containing i and j, respectively.
- $find(i)$. Given an object i, return an integer "identifying" the set containing it.

In programming jargon, we say that this is the *interface* of the data type.

Here, it is assumed that i and j are valid objects of course, i.e., $0 \leq i, j < n$. This data type is also often called "union-find" for obvious reasons.

Let's clarify what we mean by "identifying the set" first. The integer returned by $find(i)$ is supposed to identify the set containing i. For example, if i and j belong to the same set, then $find(i)$ and $find(j)$ must be the same integer. On the other hand, if they belong to different sets, then $find(i)$ and $find(j)$ must be different integers. This is the only use of these integers—they are meaningless beyond this. They can really be anything as long as they satisfy the said requirements. The integers also usually change when the sets involved are modified using *union* operations.

1.2 The Disjoint Sets data structure

Having described the *data type*, let's now turn to the *data structure*, that is, its implementation in code. We'd like to support the above two operations, as well as initializing a new instance of the data type. Thus, as a C++ `struct`, it naturally should look like this:

```cpp
struct DisjointSets {
    ... // TODO
    DisjointSets(int n) {
        ... // TODO
    }
    void onion(int i, int j) {
        ... // TODO
    }
    int find(int i) {
        ... // TODO
    }
};
```

The "`... // TODO`" parts are missing code that we'll fill in later. Also, note that we can't use the name "`union`" since it's a **keyword** in C++, just like "`for`", or "`void`", or "`struct`" etc., are keywords. Hence, we use "`onion`".

The part with "`DisjointSets(int n)`" is the constructor; it is called when the data structure is created.

1.3 Using the data structure

Before discussing how to implement the data structure, we can get more used to the idea that the data structure is truly separate from the data type by demonstrating that we can actually *use* the data structure even if we haven't coded it yet!

The key is that the data type's interface is already well-defined, so assuming everything goes to plan, and the implementation has no bugs, then we can simply *trust* that the data structure does what it's supposed to do. This is similar to the fact that a C++ `vector` is a data structure you simply *trust* when you use—you don't stop to think "what if the C++ implementors made a bug while implementing `vector`?"

Anyway, let's get used to this idea by solving the following problem:

Problem 1.1. There are n computers labelled 0 to $n-1$ on a line in that order. Initially, all computers are *inactive*.

Every day for n days, exactly one inactive computer becomes *active*. Specifically, on day i, computer s_i becomes active.

Two computers i and j are *connected* if they are both active and all computers between them are also active. Note that an active computer is always connected to itself.

At the end of each day, print the largest subset of computers that are all connected to one another.

$1 \le n \le 2 \cdot 10^5$

We can solve this *assuming the Disjoint Sets data type is available to us*. We don't need to know how it's going to be implemented! The idea is to use the Disjoint Sets data type to encode the different sets of contiguous active computers at any time—notice that these are *disjoint* sets! Thus, we can use *find* to identify whether two computers are connected or not. Then, whenever a computer becomes active, it gets *merged* to the sets to its left and right using *union*. (To be honest, solving the problem is already quite straightforward assuming that Disjoint Sets is available—I encourage you to try it yourself before moving on.)

Anyway, our program starts as follows:

```
int n; cin >> n;
DisjointSets sets(n);
set<int> active;   // set of active computers
map<int,int> size; // size[x] is the size of the "set" x.
int max_size = 0;
```

We take the input `n`, then initialize a Disjoint Sets structure with n elements. We also have a set called `active` to keep track of the currently-active computers, and the map `size` which maps each disjoint set to its size. Each key in the `size` map is an integer identifying the set—these integers are the ones returned by the *find* operations.

We now iterate through all n days. On each day, a new computer s_i becomes active. So we do the following:

```
for (int i = 0; i < n; i++) {
    int s; cin >> s;

    // s becomes active. also initialize size of the set
    active.insert(s);
    size[sets.find(s)] = 1;

    ...
}
```

Next, we merge computer `s` with its neighbors, `s - 1` and `s + 1`, but only if they're already active, hence we have the `active.count(t)` check here. We also only merge if they're not in the same set:

```
for (...) {
    ...

    // merge with neighbors if possible. neighbors are s - 1 and s + 1
    for (int t : {s - 1, s + 1}) {
        // only combine if the neighbor is active
        // AND they are in different sets
        if (active.count(t) and sets.find(t) != sets.find(s)) {
            int combined_size = size[sets.find(t)] + size[sets.find(s)];
            sets.onion(t, s);
            size[sets.find(s)] = combined_size;
        }
    }

    ...
}
```

Finally, we update the maximum size if needed—maybe the current set is larger than the previous largest set—and print the answer:

```
for (...) {
    ...

    // update max size, and print
    max_size = max(max_size, size[sets.find(s)]);
    cout << max_size << '\n';
}
```

The overall code looks like this:

```
int n; cin >> n;
DisjointSets sets(n);
set<int> active;   // set of active computers
map<int,int> size; // size[x] is the size of the "set" x.
int max_size = 0;
for (int i = 0; i < n; i++) {
    int s; cin >> s;

    // s becomes active. also initialize size of the set
    active.insert(s);
    size[sets.find(s)] = 1;

    // merge with neighbors if possible. neighbors are s - 1 and s + 1
    for (int t : {s - 1, s + 1}) {
        // only combine if the neighbor is active
        // AND they are in different sets
        if (active.count(t) and sets.find(t) != sets.find(s)) {
            int combined_size = size[sets.find(t)] + size[sets.find(s)];
            sets.onion(t, s);
            size[sets.find(s)] = combined_size;
        }
    }

    // update max size, and print
    max_size = max(max_size, size[sets.find(s)]);
    cout << max_size << '\n';
}
```

Notice that the meat of the work really is in the Disjoint Sets structure. If the data structure is correct, then this solution is clearly correct. And if the *union* and *find* operations are fast, this solution should also be fast. So in the next section, we proceed with learning how it is implemented!

Exercise 1.1. We didn't check that the neighbor `t` is within bounds, i.e., "`0 <= t and t < n`". Why is that okay here?

Also, suppose we want to use a `vector` of `bool`s for the `active` variable instead of a `set`. Explain how to update the code.

Exercise 1.2. Show that in this particular problem, we don't need to check for the condition `sets.find(t) != sets.find(s)`.

Exercise 1.3. Show how to solve the version of the problem where we want to print the *sum of the squares of the sizes of the components*, instead of the largest size.

2 Implementation

The previous section demonstrates that a data type can be trusted simply for its interface—we don't need to see its internal workings in order to use it. This is an essential skill a good problem-solving programmer needs to learn—the idea that you can separate a problem into multiple parts and then tackle each part separately.

Indeed, creating a new data type such as `DisjointSets` is like extending your programming language's library, and problem-solving consists not only of breaking down a problem into smaller constituent parts, but also *building up* your language to a level that's much easier to use. An experienced problem-solver will have learned many more data structures, some of them complicated and advanced, so from their perspective it seems like they're using a more powerful language—they can pretty much implement any of these advanced data structures whenever they need to.

Of course, to complete the circle, one still needs to be able to implement the said data structure, so in this section, we tackle how to do it with Disjoint Sets.

2.1 Straightforward Implementation

To get used to the idea of multiple implementations, let's try to implement it in the straightforward way first.

By the way, please **don't memorize the implementation here**! Even though it's "straightforward", it's not necessarily the easiest way to implement Disjoint Sets. The Disjoint Sets data type has the unusual property that the faster way is also easier to implement than the straightforward way! This is uncommon for data structures. Anyway, the point is that you don't have to completely remember how to do it this way.

However, although the straightforward way is harder, it has the advantage that it's...well, *straightforward*. There's no particular ingenuity or cleverness needed to come up with it—you just implement the requirements in the most "obvious" way. The "better" way may be easier to code, but it's not necessarily something you could come up with on your own.

So I recommend going through this simply with understanding in mind. Just read and understand the code—no need to memorize or learn how to do it yourself. Then in the next section, we will learn how to do it in the better way.

Anyway, what's this "straightforward way"? Well, an instance of the "disjoint sets" data type is just a bunch of sets, so why not implement it with, well, *a bunch of sets*?

So our `struct` will look like this:

```
struct DisjointSets {
    vector<set<int>> sets;
    DisjointSets(int n) {
        ... // TODO
    }
    void onion(int i, int j) {
        ... // TODO
    }
    int find(int i) {
        ... // TODO
```

```
    }
};
```

Here, `sets` is just a list of sets. Initially, it contains n sets, each containing a single integer. So we do that initialization at the constructor:

```
struct DisjointSets {
    vector<set<int>> sets;
    DisjointSets(int n) {
        sets.resize(n);
        for (int i = 0; i < n; i++) {
            sets[i].insert(i);
        }
    }
    ...
}
```

Here, we resized `sets` to be of size `n`. Then we made the ith set contain exactly the integer i with a simple `for` loop.

Next, to implement $find(i)$, we simply go through all the sets and look for the set containing i. What should we return? Well, it has to identify the *set*, so a good return value is the *index* of the set in the list.

```
    int find(int i) {
        for (int s = 0; s < sets.size(); s++) {
            if (sets[s].count(i)) return s;
        }
        throw runtime_error("not found!");
    }
```

It is now obvious that if i and j belong to the same set, then $find(i) = find(j)$, and if they belong to different sets, then $find(i) \neq find(j)$.

Now, what about $union(i, j)$? Well, we can simply find the sets containing i and j and then merge them by inserting the elements of one set to the other:

```
    void onion(int i, int j) {
        i = find(i);
        j = find(j);
        if (i != j) { // we only need to merge sets[i] and sets[j]
                      // if they aren't the same set

            // add all elements of sets[i] to sets[j]
            for (int v : sets[i]) sets[j].insert(v);

            // then, clear the old set sets[i]
            sets[i].clear();
        }
    }
```

Note that we cleared one of the sets (`sets[i]`)—if we don't do this, then we won't have *disjoint* sets anymore, which is pretty strange for a data type called "Disjoint Sets"!

Also, notice the "`i != j`" check here—of course, if i and j already belong to the same set, then we don't have to do anything.

That's it! We now have a Disjoint Sets data structure. It's really that straightforward! The only downside is that the running time of the operations are... pretty bad. But it's okay. The main point anyway is that it's possible to implement a data structure by simply following the specifications of the interface.

Just how bad are the running times? Well, let's look at the *find* operation. We're looping through *all* sets, and there are n sets, so the *find* operation is already at least $\Omega(n)$! And since *union* calls *find* twice, it is also at least $\Omega(n)$. And we didn't even account for the log-factor overhead of using sets! (Recall that C++ `set` operations run in $\mathcal{O}(\log n)$.) So the time complexities are pretty bad!

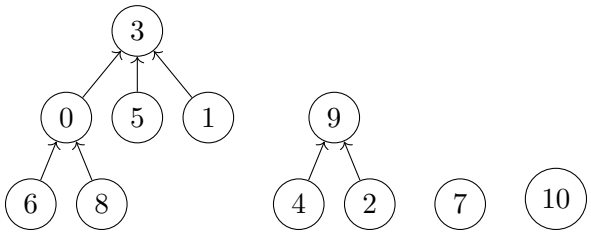
There are many ways this implementation can be improved, but we won't discuss them. Instead, we will just move on to the "better" way of implementing Disjoint Sets—via *forests*. As it turns out, they're better in multiple ways, both in terms of running time and also in *ease of implementation*, so really, there isn't much reason not to use them!

2.2 Tree-Based Implementation

We will now implement Disjoint Sets via *forests*. Recall that a forest is just a bunch of trees. In our case, a tree will naturally represent one disjoint set.

Here's how the forest could look like if the state of the Disjoint Sets is

$$[\{0, 1, 3, 5, 6, 8\}, \{2, 4, 9\}, \{7\}, \{10\}]:$$



While the drawing looks nice, it seems intimidating to implement. But looking closer, really the only thing we need to keep track of to reproduce a diagram like this is the *parent* of each node. In other words, the data structure only consists of a single array, which we'll call *parent*, where $parent[i]$ denotes the parent node of node i. For the root nodes, there are no parents, and we can use a dummy value like -1. In the example diagram, the $parent[i]$ array looks like this:

i	0	1	2	3	4	5	6	7	8	9	10
$parent[i]$	3	3	9	-1	9	3	0	-1	0	-1	-1

Initializing the forest of Disjoint Sets is now pretty straightforward. Initially, each node is in its own tree, so all $parent[i]$ values are -1:

```
struct DisjointSets {
    vector<int> parent;
    DisjointSets(int n) {
        parent.resize(n, -1);
    }
    ...
```

```
}
```

Let's now think about *find*(i). We need to identify the set—equivalently, the tree—containing i. We only have the `parent` array, and the only thing we can do is to *follow* it. At the end of the journey, we will reach the root of the tree. Well, we can simply use the root to identify the tree itself, because two nodes are in the same tree if and only if the roots of their tree are the same!

Thus, the *find* function simply looks like this, using a recursive implementation:

```
int find(int i) {
    if (parent[i] == -1) {
        return i;
    } else {
        return find(parent[i]);
    }
}
```

What about *union*(i, j)? We need to combine the trees containing i and j somehow. Let's again find the roots of the trees containing i and j. But then, if we have the roots, then combining is pretty simple too—just set one of them to be the parent of the other! Thus, *union*(i, j) can easily be implemented as follows:

```
void onion(int i, int j) {
    i = find(i);
    j = find(j);
    if (i != j) parent[i] = j; // simply set the parent of one to the other!
}
```

The two calls to `find` is to get the roots of the corresponding trees. And as before, we only merge trees if they are different—we check this using "`i != j`".

We now have a working Disjoint Sets data structure using forests! And it's much easier to implement than the "straightforward" way!

Unfortunately, the running time isn't necessarily much better. In the worst case, the trees are very, very tall, so walking up towards the root could take $\mathcal{O}(n)$ time in the worst case. But at least it's much simpler to implement!

2.2.1 Optimizations

Luckily, there are two simpler optimizations that we can do to improve the worst-case running time.

Union by height. Our first technique is somewhat natural. We don't want trees with large heights, so when merging trees, it makes sense to make the height as small as possible.

Thus, when merging trees with roots i and j, it makes sense to attach the shorter tree as a child of the larger tree, i.e., if *height*$(i) >$ *height*(j), then i will be the parent, while if *height*$(i) <$ *height*(j), then j will be the parent. If the heights are equal, then either one can be the parent.

Let's see how this is implemented. Since we now need to keep track of the heights, we make a second array which we'll call `height`, where `height[i]` denotes the height of the subtree rooted at `i`. At the beginning, every tree consists of a single node, so their heights are all 0, so

the initialization looks like this:

```
struct DisjointSets {
    vector<int> parent;
    vector<int> height;
    DisjointSets(int n) {
        parent.resize(n, -1);
        height.resize(n, 0);
    }
```

Now, here's how to update the $union(i, j)$ function:

```
    void onion(int i, int j) {
        i = find(i);
        j = find(j);
        if (i != j) {
            // set j to be the taller tree
            if (height[i] > height[j]) swap(i, j);

            // update height if needed
            if (height[i] == height[j]) height[j]++;

            // set parent
            parent[i] = j;
        }
    }
```

Notice the following:

- The line "`if (height[i] > height[j]) swap(i, j);`" ensures that `j` is the taller tree by swapping `i` and `j` if `j` is shorter.
- We increment `height[j]` iff the two trees have the same height. This is because if the trees have unequal heights, then attaching the shorter tree to the larger tree doesn't change the height.
- The last line is the same as our original implementation—we make `j` the parent (since it is taller).

Now, the code is still clearly correct. The only changes we made are to include additional logic that chooses which among `i` or `j` will be the parent, but as far as correctness is concerned, either could be the parent and it will still be correct. However, since our additional logic greedily chooses among the possibilities the one that results in a shorter tree, the worst-case of our solution could be better.

And indeed, it is! The key observation is the following: the only way to produce a tree of height h is to merge two trees of heights **exactly** $h-1$—because obviously both trees must have height less than h and one of them must have height exactly $h-1$, but if the other one has height $h-2$ or lower, then that shorter tree will be the child, and the height of the merged tree wouldn't be h. Thus, we have the following chain of reasoning:

- A tree of height 0 has at least 1 node (obviously).

- A tree of height 1 is formed from two trees of height 0. Each of them has at least 1 node, so a tree of height 1 has at least $1 + 1 = 2$ nodes.
- A tree of height 2 is formed from two trees of height 1. Each of them has at least 2 nodes, so a tree of height 2 has at least $2 + 2 = 4$ nodes.
- A tree of height 3 is formed from two trees of height 2. Each of them has at least 4 nodes, so a tree of height 3 has at least $4 + 4 = 8$ nodes.
- etc.

In general, this shows that:

Theorem 2.1. Using "union by height", a tree of height h has at least 2^h nodes.

Now, consider the tallest tree in our forest. Suppose it has height h. By this theorem, it has at least 2^h nodes. But there are at most n nodes in the tree. Therefore, $2^h \leq n$, i.e., $h = \log_2 n$. Thus, the tallest tree—and indeed every tree—has height $\mathcal{O}(\log n)$, so each operation (*union* or *find*) runs in $\mathcal{O}(\log n)$ time!

Path compression. Let's ignore union-by-height for a moment and return to our original, naive implementation of the forest, so in particular, the trees can be very tall again.

A really bad case that can happen is if we have a really tall tree that's basically a line with height $n - 1$, and we repeatedly call *find* on the bottommost node. In this case, we will repeatedly walk up the whole tree for every call, which is pretty bad. However, there's a really simple way to improve this. After the first call, why not just *point* every node along the path directly to the root? This basically breaks apart the whole path and ensures that we can't repeatedly trigger the worst case.

This technique is called *path compression*—we compress the paths we passed through.

Changing the code is pretty simple—it involves just changing one line of the `find` function:

```
1    int find(int i) {
2        if (parent[i] == -1) {
3            return i;
4        } else {
5            return parent[i] = find(parent[i]);
6        }
7    }
```

Notice that we only replaced "`return find(parent[i]);`" with "`return parent[i] = find(parent[i]);`". In other words, we're updating `parent[i]` to point to the root!

Now, what's the running time? This is much harder to analyze—you'll probably learn the proof technique in a future module about amortized analysis—but as it turns out, the running time is $\mathcal{O}(\log n)$ *amortized*. If you don't know what "amortized" means, let's rephrase this to the following: *the running time of q calls of either union or find (and initialization) is $\mathcal{O}(n+q \log n)$*. So it's pretty fast!

Union by height + path compression. Although log-ish complexity is already pretty good, we can actually do better! Notice that there's nothing stopping us from using *both* union by height and path compression!

If we do both optimizations (union by height + path compression), then of course the running time is definitely not worse than our previous estimate of $\mathcal{O}(\log n)$ per operation, but it could potentially be better. And indeed, it's much, **much** better! It can be shown that the running time of q operations (and initialization) is $\mathcal{O}((n+q)\alpha(n))$, where α is a certain **very** slow growing function. It's called the **inverse Ackermann function**, and all you need to know about it is that it's nondecreasing and that it grows very slowly, in fact so slowly that $\alpha(n) < 5$ for all numbers you'll encounter in practice. Thus, the complexity "$\mathcal{O}((n+q)\alpha(n))$" is effectively linear.

What do I mean by "numbers you'll encounter in practice"? Let me demonstrate:

- The estimated number of atoms in the universe is on the order of 10^{80}. If $n = 10^{80}$, then $\alpha(n) < 5$.
- The number 10^{100} is called a *googol*. It is 1 followed by a hundred zeroes. If $n = 10^{100}$, then $\alpha(n) < 5$.
- The number $10^{10^{100}}$ is called a *googolplex*. It is 1 followed by a googol zeroes. If $n = 10^{10^{100}}$, then $\alpha(n) < 5$. Note that there isn't even enough matter in our universe to fully write a googolplex in decimal!
- In fact, the smallest n such that $\alpha(n) \geq 5$ is much larger than $10^{10^{10^{10000}}}$—much, much larger actually!
- Even more crazily, the smallest n such that $\alpha(n) \geq 6$ is a power tower like $2^{2^{2^{2^{\cdot^{\cdot^{\cdot}}}}}}$ where the height of the tower is **gigantic**. How gigantic? Well, it's so large, that there isn't even enough matter in the universe to write just the *number of digits of the height*!

Now, I hope you're convinced that you won't encounter any n with $\alpha(n) > 5$.

Here's an implementation with both optimizations:

```
struct DisjointSets {
    vector<int> parent;
    vector<int> rank;
    DisjointSets(int n) {
        parent.resize(n, -1);
        rank.resize(n, 0);
    }
    void onion(int i, int j) {
        i = find(i);
        j = find(j);
        if (i != j) {
            if (rank[i] > rank[j]) swap(i, j);
            if (rank[i] == rank[j]) rank[j]++;
            parent[i] = j;
        }
    }
    int find(int i) {
        if (parent[i] == -1) {
            return i;
        } else {
```

```
            return parent[i] = find(parent[i]);
        }
    }
};
```

Notice that we used the word `rank` instead of `height`. This is because when doing both optimizations, the word "height" becomes somewhat inaccurate—when we compress paths in *find*, the height of the tree sometimes decreases. Of course, we don't bother actually updating the `height` array to actually contain the accurate heights—for one, that may not be easy or quick to do—so we just use a different term. And people chose the word "rank".

Note that the implementation is still pretty short, even though our runtime is already effectively linear! But we can actually shorten it a bit more if you know a bit more of C++ syntax. For example, this is how I sometimes implement it:

```
struct DisjointSets {
    vector<int> parent, rank;
    DisjointSets(int n): parent(n, -1), rank(n, 0) {}
    void onion(int i, int j) {
        if ((i = find(i)) == (j = find(j))) return;
        if (rank[i] > rank[j]) swap(i, j);
        if (rank[i] == rank[j]) rank[j]++;
        parent[i] = j;
    }
    int find(int i) {
        return parent[i] == -1 ? i : parent[i] = find(parent[i]);
    }
};
```

Note the use of the ternary operator in `find`, and notice that it still does path compression.

This shows that fundamentally, implementing Disjoint Sets with forests is conceptually really that simple!

Union by size. As a side note, you can also perform *union by size* instead of *union by height*—make the *smaller* tree (fewer nodes) the child of the one with the larger tree. With union by size, you can prove something analogous to Theorem 2.1 to show that the forest has height $\mathcal{O}(\log n)$, so each operation still takes $\mathcal{O}(\log n)$.

Exercise 2.2. Suppose we use "union by size". Let x be a non-root node and p be its parent. Show that the size of the subtree rooted at p is at least twice the size of the subtree rooted at x. Using this, show that the height of the forest is $\mathcal{O}(\log n)$.

And if we combine union by size with path compression, it can also be shown that q operations take $\mathcal{O}((n+q)\alpha(n))$ time, which is basically the same as with union by rank + path compression!

Here's one way to implement union by size + path compression:

```
struct DisjointSets {
    vector<int> parent, size;
    DisjointSets(int n): parent(n, -1), size(n, 1) {}
```

```
    void onion(int i, int j) {
        if ((i = find(i)) == (j = find(j))) return;
        if (size[i] > size[j]) swap(i, j);
        size[j] += size[i];
        parent[i] = j;
    }
    int find(int i) {
        return parent[i] == -1 ? i : parent[i] = find(parent[i]);
    }
};
```

Other optimizations. There are many other variations of these optimizations, but the above should be enough for your first encounter with union-find.

3 Miscellaneous

Let's now take a step back and discuss data structures a bit more generally.

A lot of problems require you to use data structures to solve. A data structure–powered solution generally has two parts:

- Defining the *data type* that you need;
- Implementing that data type via a *data structure*.

Note that these are two separate things—as demonstrated above, we were able to use the Disjoint Sets data type even without knowing how it's implemented! We simply assumed that it's working as expected, and used it via its interface. Another advantage of separating these two is that you can treat them as different subproblems—for example, a data type can often be implemented using different data structures (as we've seen above!), so you can consider "finding the best data structure" as its own subproblem!

There are some "meta" observations related to this. Often, the problem won't tell you what data structure to use, or even that you'll need to use a data structure in the first place. That's all part of the problem. Thus:

- To be able to use data structures for your problem, you must first be able to recognize that you'll *need* a new (or existing) data type.
- You must also have a general idea that your data type isn't so miraculous that it's impossible to implement efficiently—for this, you need both familiarity of existing standard data structures, and also some experience "inventing" new data structures.
- Sometimes, a data type can be implemented with multiple data structures, and the running times may not necessarily be comparable with each other. For example, suppose you need a data type that supports queries and updates. Then maybe one data structure does queries more efficiently than updates, and another does updates more efficiently than queries. Thus, neither is strictly "better" than the other, and you use whichever is better for your current use case. For example, if there are more queries than updates, then it might be better to use the former.

  In fact, sometimes you may want to do a hybrid approach, where you use one data structure or the other depending on the ratio between queries and updates!
- On the other side of the problem-solving process, you must also make sure that the operations you're using to solve the problem at a high level (and that you're planning to delegate to your data type) are "simple" enough that it's conceivable for there to be a data structure that handles them efficiently. For example, incrementing all values in a range is clearly simpler than taking the square roots of the numbers in the range, and it's much easier to convince oneself that the former has a better chance of being able to be handled with a data structure than the latter.

These "meta" observations are a bit abstract, and some of them might be more useful to you than others. The most effective way to learn them is through lots of practice (and asking lots of questions to coaches and more experienced participants!).

3.1 Inventing a new data structure

But how do you "invent" a data structure in the first place? For really simple data structures, you may be able to get by with just memorizing the standard ones, but sooner or later you'll encounter a problem where you'll have to *invent* a data structure ad-hoc, one whose sole purpose is to solve the problem at hand. Again it boils down to experience, but there are a few general guidelines to doing it effectively.

First, it's not sufficient to describe your structure *vaguely*. Well, that's okay if you're still just coming up with the general idea, but eventually you'll have to define your data structure more precisely. For example, what does it look like? What properties does it satisfy? What are the things you're keeping track of? What do the variables mean? What properties do the variables satisfy?

You can then perform queries by simply *assuming* that these properties hold—for example, if one of the properties is that "the height is $\mathcal{O}(\log n)$", then you may be able to safely traverse a path from root to leaf and be sure that it's still efficient. Or if one of the properties is "the values at the left subtree are all less than the root" (and something similar for the right subtree), then you may be able to do something similar to binary search to find an item.

Now, when performing updates, we have to take care to ensure that the properties still hold. This is the crux of data structures—maintaining the *invariants* and properties of the tree, while doing as small amount of work as you can to do so. For example, maybe in your structure, you're declaring that some of your nodes are "black" and some are "white", and you have the requirement that "no two white nodes are adjacent." Then when deleting a black node for example, you must take care that the requirement won't be violated—you may have to do more manipulations just make sure of that. Thus, the question you should be asking is "what is the minimum amount of changes needed to maintain the invariants of my data structure?"